Table of Contents

Wat zijn agents? *2*

Belangrijke kenmerken van agents **2**

Het ReAct-pattern **2**

Hoe werken Agents samen *4*

Wat is een Multi-agent systeem (MAS)? **4**

Welke samenwerkingspatronen (patterns) bestaan er? **4**

Hoe context / state gedeeld wordt tussen agents **7**

Conflict- en consensus afhandeling **10**

Concrete voorbeelden **11**

Multi-agent frameworks *11*

LangGraph **11**

Architectuur 11
Memory 12
Beslissingen nemen en tools aanroepen 12
Tekortkomingen 12
Use cases 13

Semantic Kernel **14**

Kernel 14
Automatic Function Calling 14
Plugins 14
Connectors 14
Memory 15
Tekortkomingen 16
Use cases 16

Vergelijking tussen Semantic Kernel en LangGraph **17**

Mcp server/client architectuur *17*

Componenten **18**

Communicatie flow **18**

Tools **19**

MCP: Resources, Tools en Prompts **19**

Resources 19
Tools 19
Prompts 20

Onderliggende communicatie protocollen **20**

Gevaren **20**

Hoe het MCP protocol in multiagentsystemen werkt **21**

Bronnen: *22*

Wat zijn agents?

In de context van moderne taalmodellen zijn agents software-componenten die autonoom een taak kunnen uitvoeren namens een gebruiker of applicatie. Een agent combineert een LLM met bijkomende mogelijkheden zoals tool-gebruik, geheugen, planning en contextbeheer. In plaats van enkel tekst te genereren op basis van een prompt, beslist een agent welke acties nodig zijn, welke tools moeten worden aangeroepen en hoe informatie uit eerdere stappen moet worden verwerkt. Dit maakt agents geschikt voor inzet in bedrijfsomgevingen waar meerdere databronnen, systemen en workflows moeten worden geïntegreerd.

Belangrijke kenmerken van agents

Volgens Google Cloud (2024) beschikken moderne AI-agents over een reeks fundamentele kenmerken die hun intelligent gedrag en autonomie bepalen. Oorspronkelijk lag de nadruk op reasoning (redeneren) en acting (handelen), zoals beschreven in het ReAct-framework, maar de mogelijkheden van agents zijn sindsdien uitgebreid met bijkomende eigenschappen:

- **Reasoning (redeneren)**: De agent voert interne redenering uit op basis van de huidige context. Dit gebeurt doorgaans via ReAct-achtige patronen, waarbij het model nadenkt over de volgende actie, tools selecteert en complexe beslissingen opdeelt in subtaken.

- **Acting (handelen)**: De agent kan acties uitvoeren via tools of externe systemen. Dit omvat API-calls, databasequeries, MCP-toolinvocaties of integraties met bedrijfsapplicaties. Acting is de essentie van praktische inzet: zonder tools blijft een agent enkel tekst genereren.

- **Observing (waarnemen):** het verwerken van terugkoppeling (tool output, resultaten van andere agents, status-updates).

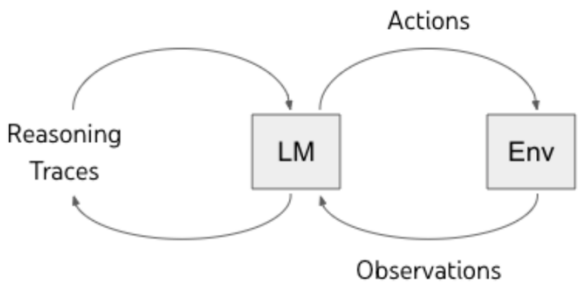
- **Planning (plannen):** het strategisch plannen van stappen om doelen te bereiken. Agents met planningsvermogen evalueren mogelijke acties, anticiperen op toekomstige toestanden en kiezen de optimale aanpak.

- **Collaborating (samenwerken):** het vermogen om effectief met andere agents of mensen samen te werken om een gemeenschappelijk doel te bereiken. Dit vereist communicatie, coördinatie en inzicht in elkaars perspectieven.

Het ReAct-pattern

Het ReAct-pattern *(Reason + Act)* ligt aan de basis van hoe moderne LLM-gebaseerde agents nadenken over een situatie en daarop doelgericht handelen. Het werd geïntroduceerd door Google Research in de paper *"ReAct: Synergizing Reasoning and Acting in Language Models"* (Yao et al., 2022) en combineert redeneren, handelen en observeren in één cyclisch proces. Waar traditionele taalmodellen enkel tekst genereren, stelt ReAct een agent in staat om zelfstandig beslissingen te nemen, acties uit te voeren (zoals tool- of API-aanroepen) en zijn interne toestand te actualiseren op basis van feedback uit de omgeving.

De ReAct-cyclus verloopt volgens het patroon:



- **Thought (denken):** de agent redeneert intern over de huidige context, zijn doel en mogelijke volgende stappen.
- **Action (handelen)**: de agent voert een actie uit bijvoorbeeld een query naar een database of het aanroepen van een externe tool.
- **Observation (refrecteren)**: de agent ontvangt feedback of resultaten uit de omgeving en gebruikt die informatie om zijn interne toestand aan te passen.

Het ReAct-patroon (Reason + Act) stelt agents in staat om niet uitsluitend te vertrouwen op hun interne kennis, maar ook actief externe informatie op te vragen via tools of communicatie met andere agents. Hierdoor neemt het risico op hallucinaties af, terwijl collaborative reasoning wordt gestimuleerd: agents kunnen informatie delen of ophalen om tot beter onderbouwde beslissingen te komen.

Daarnaast impliceert het ReAct-patroon een vorm van state tracking. Een agent moet kunnen onthouden wat hijzelf en andere agents in eerdere stappen hebben gedaan. Dit gebeurt via interne geheugens (zoals contextbuffers of vectorstores) of via gedeelde geheugensystemen, bijvoorbeeld blackboard architectures. Op die manier blijft de redenering van de agent consistent over meerdere stappen heen.

In sommige toepassingen, zoals supervised multi-agent systems, kan ook menselijke tussenkomst plaatsvinden. Een menselijke operator kan de reasoning traces van een agent inspecteren en waar nodig bijsturen, wat zorgt voor gecontroleerde samenwerking tussen autonome agents en menselijke supervisie.

Het ReAct-pattern vormt daarmee het fundamentele gedrag van moderne LLM-agents. Door reasoning en actie in één iteratieve cyclus te combineren, kunnen agents niet alleen nadenken over een probleem, maar ook gericht externe informatie opvragen, tools aanroepen en hun interne context bijwerken op basis van de ontvangen feedback. Dit maakt het model geschikt voor scenario’s waarin meerdere stappen, afhankelijkheden en databronnen samenkomen.

Frameworks zoals Semantic Kernel en LangGraph bouwen rechtstreeks op dit principe: tools worden geactiveerd tijdens de Act-fase, resultaten worden verwerkt in Observation, en reasoning-traces sturen de volgende stap in de workflow. In multi-agentarchitecturen vormt deze ReAct-loop het gemeenschappelijke mechanisme waarmee agents samenwerken, context delen en via orchestratie tot een gecoördineerd eindresultaat komen.

Hoe werken Agents samen

Wat is een Multi-agent systeem (MAS)?

Een multi-agent systeem (MAS) bestaat uit meerdere autonome agents die elk een deel van een groter probleem behandelen. In plaats van één enkel model dat alle taken uitvoert, wordt het werk verdeeld over gespecialiseerde agents met eigen rollen, kennis en verantwoordelijkheden. Ze communiceren en coördineren hun acties om een gemeenschappelijk doel te bereiken. Daardoor kunnen ze complexere taken verwerken dan één enkel systeem.

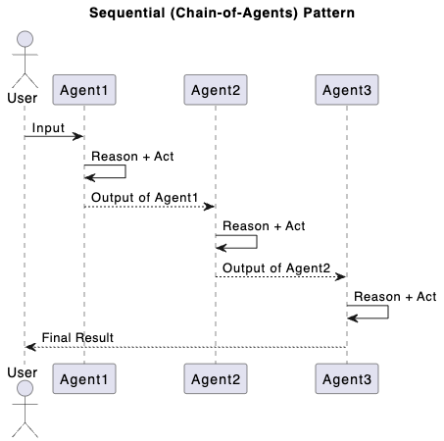
Elke agent volgt hierbij het ReAct-principe: hij redeneert over de context, voert acties uit via tools en verwerkt de feedback om zijn volgende stap te bepalen.
In een MAS wisselen agents informatie uit en stemmen ze hun acties op elkaar af. Dit maakt het mogelijk om processen met meerdere stappen, afhankelijkheden of verschillende databronnen te behandelen. Een agent kan bijvoorbeeld externe data ophalen, een tweede agent kan die data analyseren, en een derde kan een rapport of besluitvorming genereren. Zo ontstaat een schaalbare structuur waarin agents parallel werken, subtaken delegeren en elkaar aanvullen.

Welke samenwerkingspatronen (patterns) bestaan er?

Naarmate taalmodellen en multi-agent systemen complexer worden, is het belangrijk om ze betrouwbaar te laten functioneren. Dit kan door gebruik te maken van verschillende orchestratiepatronen die de samenwerking tussen agents structureren en coördineren.
Dit zijn architecturale patronen, hoe agents samenwerken op hoog niveau.

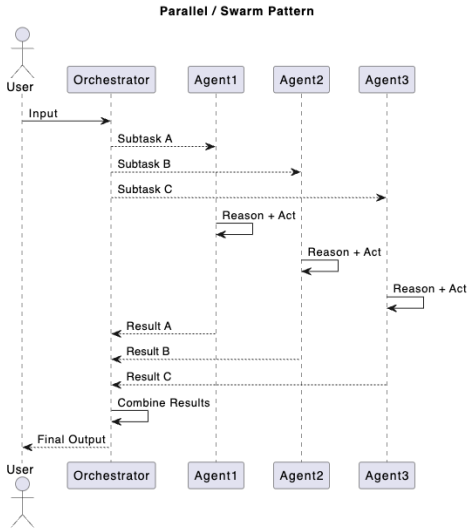
1. Sequential (Chain-of-Agents) Pattern

- **Beschrijving**: Agents worden agents sequentieel aangeroepen. De uitvoer van een agent vormt telkens de invoer voor de volgende, waardoor een lineaire keten ontstaat. Dit is geschikt voor taken die een duidelijke stap-voor-stapverwerking nodig hebben en waarbij elke stap afhankelijk is van het resultaat van de vorige.
- **Gebruik**: lineaire workflows zoals documentverwerking, data-extractie of besluitvorming-pipelines of processen die niet kunnen worden opgesplitst in parallelle subtaken.
- **Sterktes:** eenvoudig te implementeren, goed controleerbaar en transparant. Elke stap in de reasoning chain is eenvoudig te volgen.



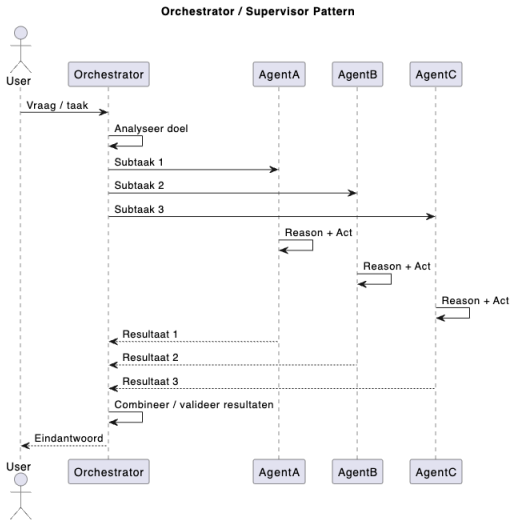
2. Parallel / Swarm Pattern

- **Beschrijving:** Bij het parallel- of swarm-patroon wordt een probleem opgesplitst in meerdere onafhankelijke subtaken. Verschillende agents voeren deze subtaken gelijktijdig uit op basis van hetzelfde doel of dezelfde input. Dit verkort de totale uitvoeringstijd en maakt het systeem meer resilient: wanneer één agent faalt, kunnen andere nog steeds hun deel van het werk leveren.
- **Gebruik:** informatie-aggregatie, ensemble reasoning, scenario-analyse, of elke workflow waarbij oplossingen tegelijk kunnen worden gegenereerd en nadien gecombineerd.
- **Sterktes:** hogere prestaties, robuustheid en fouttolerantie. De parallele uitvoering maakt het mogelijk om meerdere berekeningen simultaan of meerdere keren uit te voeren, wat de kwaliteit en betrouwbaarheid van het eindresultaat kan verbeteren.



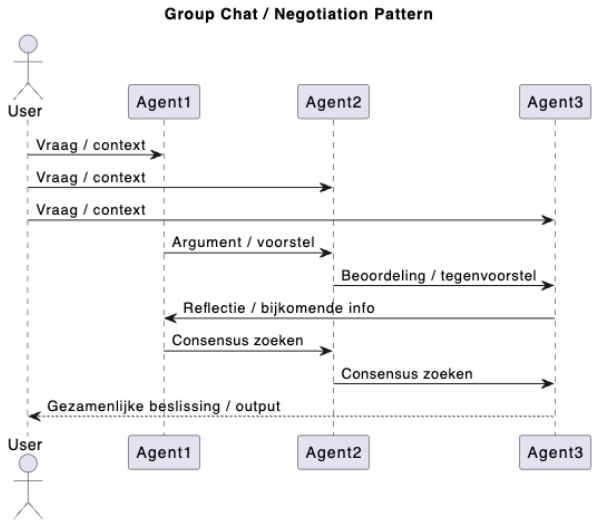
3. Orchestrator / Supervisor Pattern

- **Beschrijving:** In dit patroon stuurt één centrale agent (de orchestrator of supervisor) het volledige proces aan. De orchestrator beslist welke gespecialiseerde agents ingeschakeld worden, verdeelt subtaken, bewaakt afhankelijkheden en combineert alle resultaten tot één eindantwoord. Elke sub-agent is verantwoordelijk voor een afgebakende taak zoals data-extractie, analyse, validatie of communiceren met een externe databron.
- **Gebruik:** Workflows waarbij een duidelijke centrale sturing nodig is: systemen die meerdere agents moeten aanroepen in een niet-op-voorhand bepaalde volgorde, afhankelijkheden moeten beheren, of resultaten moeten combineren en valideren.
- **Sterktes:** Hoge controleerbaarheid en governance. Voorspelbaar gedrag, duidelijke taakverdeling en auditbaarheid. Ook is dit patroon makkelijk uitbreidbaar zonder de bestaande structuur veel te moeten aanpassen. De orchestrator kan dienen als centrale plek voor state en context.



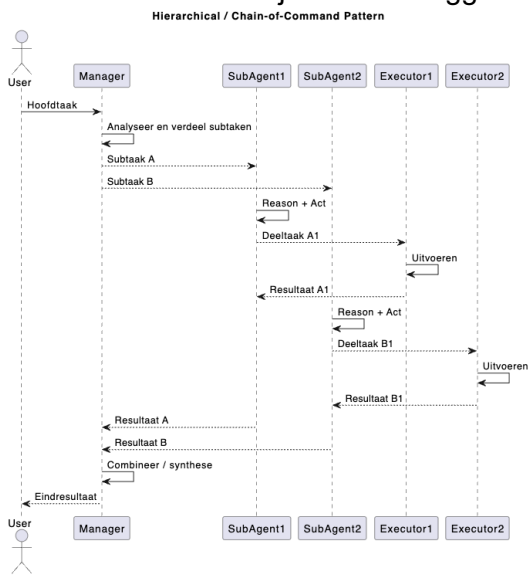
4. Group Chat / Negotiation Pattern

- **Beschrijving:** Hier communiceren agents rechtstreeks met elkaar via peer-to-peer berichten. Ze delen tussenresultaten of voorstellen en proberen via conversatie tot een gezamelijk besluit te komen. De interactie tussen de agents is minder hiërarchisch en meer open: elke agent kan initiatief nemen of reageren op anderen.
- **Gebruik:** Dit is vooral toepasbaar wanneer meerdere perspectieven of redeneringen gecombineerd moeten worden, zoals bij brainstormsessies, consensusvorming of evaluatie.
- **Sterktes:** Zeer flexibel en natuurlijk voor LLM's. Het stimuleert emergent gedrag doordat agents soms elkaars ideeën versterken, uitdagen of combineren. (Moeilijk beheerbaar?)



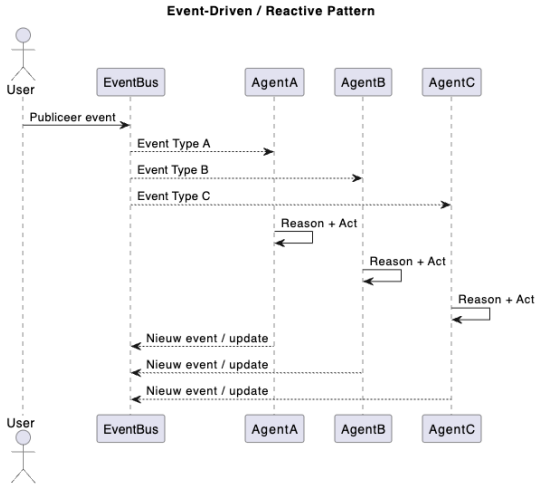
5. Hierarchical / Chain-of-Command Pattern

- **Beschrijving:** Hier worden agents georganiseerd in verschillende lagen, vergelijkbaar met een manager-structuur. Een bovenliggende agent verdeelt een complex probleem in subtaken en wijst deze toe aan gespecialiseerde sub-agents. Deze kunnen op hun beurt opnieuw lagere executor-agents inschakelen. Elke laag werkt op een verschillend abstractieniveau: van strategisch tot operationeel.
- **Gebruik:** Situaties waarin een probleem eerst moet worden opgesplitst in deel problemen (task decomposition). Dit kan voorkomen in grote workflows met meerdere niveaus, zoals meerlaagse documentverwerking, planning-taken of systemen die strategische beslissingen combineren.
- **Sterktes:** Zeer schaalbaar door de verticale taakverdeling. Nieuwe agents kunnen makkelijk toegevoegd worden op de juiste laag. Goede controle door duidelijke verantwoordelijkheden te leggen bij bepaalde agents.



6. Event-Driven / Reactive Pattern

- **Beschrijving:** In het event-driven patroon reageren agents niet op directe oproepen maar op gebeurtenissen die binnenkomen via een event bus of message queue. Iedere agent luistert naar specifieke eventtypes en voert acties uit zodra een bepaald event binnenkomt. De interactie verloopt dus asynchroon en agents weten niet wie de events genereert, enkel dat ze erop moeten reageren.
- **Gebruik:** monitoring, autonome workflows, IoT-systemen, event-gebaseerde datapipelines, automatische triggers of workflows die moeten reageren op bepaalde updates. Geschikt voor situaties waarin bepaalde acties ondernomen moeten worden zodra een bepaalde gebeurtenis voorkomt, zonder centrale aansturing.
- **Sterktes:** Zeer schaalbaar en robuust door losgekoppelde communicatie. Agents kunnen onafhankelijk van elkaar aangepast worden en ze kunnen falen zonder andere agents te blokkeren.



Hoe context / state gedeeld wordt tussen agents

Wanneer meerdere agents samenwerken, moet informatie zoals tussenresultaten, beslissingen, geheugen of parameters correct tussen hen gedeeld worden. Afhankelijk van het gekozen orchestratiepatroon kan die contextdeling sterk verschillen. Hieronder staan de belangrijkste mechanismen die gebruikt worden voor contextdeling tussen agents.

1. Prompt Chaining / sequentiële overgave van context

- Bij prompt chaining wordt de uitvoer van één agent rechtstreeks doorgegeven als invoer aan de volgende. De context wordt dus lineair doorgegeven in de keten van agents.
    - Agent A produceert output + relevante context
    - Agent B ontvangt deze informatie en gaat verder
    - Dit herhaalt zich tot het proces klaar is

- **Voordelen:** Eenvoudig, transparant, ideaal voor lineaire workflows.

- **Nadelen:** Beperkte flexibiliteit, agents kunnen niet terugkeren of parallel werken, geen gedeelde globale state.

- **Toepassing:** Documentverwerking, extract-transform pipelines, wanneer analyse in opeenvolgende stappen nodig is.

2. Gedeeld geheugen

   - Hier werken agents met een gedeeld geheugenobject of state data als gemeenschappelijke waarheid dient binnen een sessie.
     - Agent A schrijft naar “state[“key”]”
     - Agent B leest die waarde later
     - De state kan sessiegebonden of persistent zijn.

   - **Voordelen:** Agents kunnen onafhankelijk van elkaar werken maar toch beschikken ze over een gemeenschappelijk beeld van de workflow en (tussen)resultaten.

   - **Nadelen:** Dit vergt duidelijke regels over lees- en schrijfrechten, mogelijk risico op inconsistentie of race condities.

   - **Toepassing:** Systemen waar meerdere agents na elkaar of parallel beslissingen moeten nemen op basis van gedeelde informatie.

3. LLM-Driven Delegation (Agent Transfer)

   - Hier beslist een agent autonoom op basis van LLM-reasoning om de taak over te dragen aan een andere agent. De orchestratie wordt dus door de agents zelf gestuurd.
     - Agent genereert een functieaanroep om over te schakelen naar een andere agent
     - MAS-systeem onderschept dit
     - De uitvoercontext wordt verplaatst naar de doel-agent
     - Deze gaat verder waar de vorige gestopt is

   - **Voordelen:** Zeer dynamisch en contextgevoelig, agents kunnen autonomie tonen in taakrouting.

   - **Nadelen:** Moeilijker te debuggen of te voorspellen, de LLM bepaalt zelf welke agent geschikt is.

   - **Toepassing:** Hiërarchische of flexibele MAS-systemen waarbij routing afhankelijk is van inhoudelijke beslissingen.

4. Explicit Invocation / AgentTool

   - Een agent kan een andere agent rechtstreeks oproepen alsof het een tool of functie is. De aanroep is op voorhand geconfigureerd en aangemaakt. (Dus niet dynamisch zoals bij delegation)
     - Doel-agent wordt verpakt als een tool
     - Parent-agent kan deze tool aanroepen
     - Framework start sub-agent en geeft het resultaat terug

   - **Voordelen:** Gecontroleerd, voorspelbaar en eenvoudig te reproduceren.

   - **Nadelen:** Minder flexibel dan agent transfer: alle aanroepen moeten vooraf gedefinieerd worden.

   - **Toepassing:** Workflows waar bepaalde subtaken door vaste sub-agents moeten worden gedaan (validatie, planning, data-extractie).

5. Group Chat / Peer Messaging Pattern

   - Agents communiceren via conversatieberichten. De context is hier niet een apart geheugenobject, maar de opeenvolgende berichten zelf.
     - Agent 1 post een bericht
     - Agent 2 reageert met eigen bijdrage
     - De conversatielog fungeert als contextbuffer

   - **Voordelen:** Natuurlijk, flexibel, ideaal voor brainstormen en consensusvorming.

   - **Nadelen:** Weinig controleerbaar of voorspelbaar; moeilijk te reproduceren door emergent gedrag.

   - **Toepassing:** Creatieve of onderhandelingsgerichte scenario's.

6. Blackboard / Shared Knowledge Repository Pattern

   - Agents communiceren indirect via een "blackboard". Elke agent kan hier tussenresultaten, hypotheses, metadata of observaties op plaatsen. Andere agents lezen dit, vullen het aan of reageren hierop. In tegenstelling tot shared memory, vormt blackboard een actieve werkruimte in plaats van gewoon een data-opslag.
     - Agent A plaatst informatie op blackboard (bv hypothese, analyse)
     - Agent B leest de nieuwe informatie, voert eigen reasoning uit en vult aan.
     - De samenwerking is volledig losgekoppeld en asynchroon
     - Extra kan er ook nog een monitorcomponent tussen gezet worden die bepaalde agents activeert op basis van welke informatie op het blackboard staat of gezet wordt.

   - **Voordelen:** Losgekoppelde samenwerking, flexibel en makkelijk schaalbaar.

   - **Nadelen:** Vereist goed beheer van toegang, updates en versiebeheer, kan dus snel complex worden. Er moet ook gezorgd worden dat agent informatie op dezelfde manier interpreteren.

   - **Toepassing:** Complexe orchestraties zoals planning of onderzoeksproblemen. Scenario's waar kennis deel per deel wordt opgebouwd door meerdere stappen door verschillende agents.

Conflict- en consensus afhandeling

Wanneer meerdere agents in parallel of in dialoog werken, kunnen conflicten ontstaan: verschillende agents die met andere voorstellen komen, overlappende plannen of race-condities. Daarom bestaan er strategieën om deze problemen op te lossen.

1. Centrale coördinatie

   - Een centrale orchestrator of supervisor beslist welke agent welke taak krijgt, bewaakt afhankelijkheden en lost conflicten op door expliciet de prioriteit of volgorde te bepalen.
     - Supervisor ontvangt resultaten van sub-agents
     - Bepaalt welke output geldig/relevant is
     - Stuurt agents bij of herverdeelt taken

   - **Wanneer**: Geschikt wanneer agents gespecialiseerde rollen hebben, taken afhankelijkheden bevatten of wanneer voorspelbaarheid en overzicht belangrijk zijn.

   - **Typisch patroon**: orchestrator/supervisor pattern.

2. Negotiation / Dialogue tussen agents

   - Agents communiceren rechtstreeks via berichten, delen voorstellen of intenties, en komen via dialoog tot een gezamenlijke oplossing.
     - Agent A doet voorstel
     - Agent B reageert, corrigeert, voegt argumentatie toe
     - Ze herhalen dit totdat een oplossing ontstaat (emergent consensus)

   - **Wanneer**: wanneer het nuttig is om meerdere perspectieven te bekijken voor een bepaald probleem, wanneer verantwoordelijkheden tussen agents overlappen. Wanneer de oplossing niet eenduidig is en agents elkaar kunnen aanvullen.

   - **Typisch patroon**: Group chat / negotiation pattern.

3. Voting / Consensus mechanismen

   - Meerdere agents leveren een voorstel, score of voorspelling, en een aggregatiemechanisme bepaalt de beste keuze. Dit hoeft niet onmiddellijk stemmen te zijn: het kan ook een statistische of heuristische combinatie zijn.
     - Agents produceren onafhankelijk outputs
     - Systeem combineert deze (meerderheid, scoring, weighted votes)
     - Beste of meest consistente resultaat wordt gekozen

   - **Wanneer gebruiken:** Scenario's waar veel gelijktijdige outputs zijn of wanneer er individuele fouten kunnen voorkomen.

   - **Typisch patroon**: Parallel / swarm execution.

Concrete voorbeelden

1 Chain-of-Agents: (Sequential) Pattern + Prompt Chaining + Centrale Coordinatie

- Werking: Agent A leest en extraheert tekst, Agent B vat samen, Agent C genereert rapport.
- Orchestratiepatroon: Sequential, lineaire keten van agents.
- Contextdeling: Prompt chaining, elke output wordt de input voor de volgende.
- Conflict / afhakelijkheid: Centrale orchestrator bewaakt de volgorde en beslist of een stap moet worden hernomen bij fouten.
- Gebruik: Automatische rapportgeneratie, juridische documenten analyseren.

2 Parallel / Swarm pattern + Shared Memory + Voting

- Werking: Verschillende agents gebruiken elk een ander ML-model (neuraal netwerk, regressie, decision tree) om een voorspelling te doen.
- Orchestratiepatroon: Parallel, meerdere agents werken tegelijk aan dezelfde taak.
- Contextdeling: Alle voorspellingen worden opgeslaan in shared memory.
- Conflict / afhankelijkheid: Voting-mechanisme combineert de resultaten, door bv meerderheid of gemiddelde.
- Gebruik: beursanalyse, risicoberekening.

Multi-agent frameworks

LangGraph

LangGraph is relevant omdat het een framework is dat complexe agent-interacties structureert aan de hand van expliciete graafgebaseerde workflows. Het dwingt een deterministische en stateful orchechestratie op waarin elke stap, transitie en conditie volledig controleerbaar is. Dit sluit aan bij behoeften van bedrijfsapplicaties, waar voorspelbaarheid en governace essentieel zijn, zoals in kader van GDPR of risico management.

Het biedt een duidelijke hiërachische structuur waarbij een centrale orchestrator-agent via een graaf beslist welke agents worden aan gesproken en hoe hun output wordt geconbineerd.

Architectuur

LangGraph is gebouwd bovenop LangChain en laat ontwikkelaars toe om een flow van agents of stappen te definiëren.
Agents vormen de beslissingslaag bovenop de LLM. Ze lezen de state, kiezen de juiste tool of vervolgstap, en genereren de uiteindelijke output

Een Node vertegenwoordigt een stukje logica of een agent (bijvoorbeeld een LLM-call, een tool, of menselijke input). Edges verbinden nodes met elkaar en bepalen de uitvoervolgorde. Conditional edges laten toe dat de volgende stap afhankelijk is van de output of state (if-then-logica). Meerdere nodes en edges samen wordt dan een graph genoemd.

De State houdt alle context bij: input, output, tussenresultaten en variabelen. Dit maakt de flow herbruikbaar, controleerbaar en geschikt voor complexe beslissingsstructuren.
Tools zijn externe functies of systemen die de agent kan oproepen, zoals API-calls, databases of uitvoerbare code.

De tools die gebruikt worden, worden door de agent geselecteerd op basis van de instructies in de prompt.

Tools worden gekoppeld via standaard LangChain tool-interfaces. Een agent-node kan tools declareren die beschikbaar zijn in een bepaalde context. Wanneer de LLM tijdens reasoning een tool-caal genereert, onderschept LangGraph deze call en voert hij de bijhorende tool-functie uit. De output wordt dan opgeslagen in de graph state, waarna de flow verder gaat op basis van de verdere edges en condities.

Ook worden interrupts ondersteund. Deze laten een graph tijdelijk pauzeren om te wachten op externe input, zoals menselijke feedback of goedkeuring. De huidige state wordt opgeslagen via de checkpointer, waarna de graph later kan worden hervat. Dit maakt human-in-the-loop workflows mogelijk, zoals goedkeuringen, correcties of inputvalidatie.

Daarnaast ondersteunt LangGraph streaming output en het orkestreren van meerdere agents binnen één graph, wat samenwerking tussen gespecialiseerde agents mogelijk maakt.

Memory

Memory is het mechanisme waarmee LangGraph context en kennis bewaart over meerdere stappen of sessies.
Er worden twee hoofdcategorieën onderscheiden:

Short-term memory (thread-scoped). Dit type geheugen houdt de lopende conversatie of sessie bij. De informatie wordt opgeslagen in de state van de agent en bewaard via checkpoints. Short-term memory bevat onder meer berichten, context en tijdelijke bestanden, en wordt automatisch bijgewerkt bij elke nieuwe stap in de graph.

Long-term memory (cross-thread). Dit geheugen bewaart informatie die sessie-overstijgend is, zoals gebruikersprofielen, voorkeuren of historische data. De gegevens worden opgeslagen in datastores met namespaces en keys, vergelijkbaar met mappen en bestandsnamen.
Binnen deze categorie worden drie subtypes onderscheiden:

- Semantic memory: bevat feiten of kennis die de agent geleerd heeft.
- Episodic memory: slaat ervaringen of gebeurtenissen uit eerdere interacties op.
- Procedural memory: omvat regels of instructies die bepalen hoe de agent taken uitvoert.

LangGraph bewaart deze gegevens doorgaans als JSON-documenten binnen een memory store, wat consistente en herbruikbare context mogelijk maakt.

Beslissingen nemen en tools aanroepen

Agents in LangGraph volgen het ReAct-patroon bij het nemen van beslissingen.
De agent beslist telkens op basis van de huidige state en context welke actie of tool het meest geschikt is, waarbij de state na elke iteratie wordt bijgewerkt. Dit proces maakt LangGraph geschikt voor complexe beslissingsflows met meerdere tools en conditionele logica.

Tekortkomingen

LangGraph functioneert in de kern als een workflow-orchestration tool: een systeem dat context tussen nodes doorgeeft en daardoor gestructureerde, pauzeerbare en hervatbare processen mogelijk maakt. De kracht van het framework zit in het expliciete beheer van state en controleflow, maar precies daardoor ontbreekt echte autonomie. Alle stappen, takken en transitions worden volledig door de developer vastgelegd in de graph. Het model creëert zelf

geen doelen, voegt geen nieuwe stappen toe en herstructureert zijn workflow niet. Beslissingen die binnen de graph door een LLM worden genomen blijven beperkt tot de vooraf gedefinieerde ruimte die jij hebt ontworpen. LangGraph onderscheidt zich dus vooral in scenario's waar voorspelbare, duidelijk gecontroleerde workflows nodig zijn.
Daarnaast blijft het framework sterk afhankelijk van LangChain. De abstractions, tool-interfaces, message-structuren en het onderliggende ecosystem waarop LangGraph voortbouwt zijn bijna volledig afkomstig uit LangChain. Daardoor erft het niet alleen de sterktes van LangChain, maar ook de beperkingen. Het is dus geen standalone agentic systeem, maar eerder een gecontroleerde workflow-laag bovenop de bestaande LangChain-architectuur.

Extra:
- Debugging van LLM-nodes is moeilijk door niet-deterministische output.

Steile leercurve bij graph-based orchestratie, complexer dan klassieke LangChain agents.
- Checkpointing en state management vereisen extra infrastructuur (persistent storage).
- Performance bottlenecks door het sequentiële karakter van graphs (beperkte paralleliteit).
- LLM calls in decision nodes maken flows traag en duur.
- Memory-store is generiek maar niet geoptimaliseerd voor grootschalige retrieval (beperkt vergeleken met echte vector DB's).

Use cases

Zoals eerder gezegd is LangGraph vooral geschikt voor scenario's waar controle, reporoduceerbaarheid en samenwerking tussen agents belangrijk zijn. Hieronder staan een paar voorbeelden van waar LangGraph streeft.

- Gestructureerde multi-agent pipelines

LangGraph ondersteunt hiërarchische agentarchitecturen waarbij een orchestrator subagents aanstuurt voor taken zoals extractie, validatie, analyse of synthese. Dit past bij bedrijfsscenario's die voorspelbare en controleerbare samenwerking vereisen.

- Integraties met bedrijfsdata en applicaties

Via tools kunnen agents gegevens ophalen of manipuleren uit databases, API's, SharePoint of CRM-systemen. De state zorgt ervoor dat context behouden blijft over meerdere stappen heen.

- Conditionele beslissingsflows

Complexe beslissingen kunnen worden gemodelleerd via conditionele edges, waardoor workflows automatisch een passend pad volgen (bv. kwaliteitscontrole, exception handling).

- Human-in-the-loop processen

Interrupts maken het mogelijk om workflows tijdelijk te pauzeren voor menselijke validatie of goedkeuring, wat essentieel is in risicogevoelige bedrijfsprocessen.

- Auditeerbare en fouttolerante workflows

Checkpoints en persistente state zorgen ervoor dat elke stap reproduceerbaar is en dat workflows veilig kunnen worden hervat na fouten of externe storingen.

Semantic Kernel

Een lightweight, open-source development kit dat toelaat makkelijk AI agents te maken en de nieuwste ai modellen te integreren in je codebase.
Semantic Kernel combineert prompts met bestaande api's om acties uit te voeren.

Kernel

De kernel is het centrale coördinatiepunt binnen Semantic Kernel. Hier worden alle AI-modellen, plugins en connectors geregistreerd en beheerd. De kernel bepaalt welke modellen beschikbaar zijn, welke functies kunnen worden aangeroepen en hoe tools, memory en planning samenkomen tijdens de uitvoering van een agent.
Daarnaast biedt de kernel ondersteuning voor configuratie, logging en telemetry, zodat AI-agents controleerbaar, traceerbaar en betrouwbaar kunnen draaien in productieomgevingen.

Automatic Function Calling

Vroeger werd er gebruik gemaat van planners om vanuit natuurlijke taal een sequentie van functies te genereren. Deze zijn volledig verwijderd en vervangen door automatic function calling. Het moderne mechanisme waarmee een LLM zelf stap-voor stap beslist welke functies / plugins uitgevoerd moeten worden.

Dit bestaat uit een iteratieve loop:
LLM beslist om functie aan te roepen -> Semantic kernel agent voert de fucntie uit -> resultaat gaat terug naar de LLM -> de LLM beslist de volgende stap. Dit proces wordt herhaald totdat het doel bereikt wordt.

Hierdoor ontstaat een dynamische planning zonder vooraf gedefineerde workflows. De agent bepaald zelf welke acties nodig zijn. Hierbij wordt ook parallel function calling ondersteund, waardoor meerdere functies gelijktijdig kunnen worden uitgevoerd.

Plugins

Plugins maken het mogelijk om bestaande API-functionaliteit of code te koppelen aan de AI-agent zodadt deze acties kan uitvoeren die een model op zichzelf niet zou kunnen.
Dit kan native code zijn of functies uit een OpenAPI-secificatie of MCP-server.

Elke plugin bevat een beschrijving van het doel, paramaters en mogelijke effecten.
De LLM heeft deze data nodig om via automatic function calling te beslissen welke functie wanneer moet worden uitgevoerd.

2 belangrijke taken:
Data retrieval: bv documenten ophalen voor RAG
Task automation: bv api-calls, berekeningen, workflow stappen

De developper registreert plugins in de kernel, waarna de LLM autonoom beslist welke functies aan te roepen binnen een interactie.

Connectors

Connectors vormen de abstractielaag waarmee Semantic Kernel agents kunnen communiceren met externe systemen. Ze koppelen de kernel aan AI-modellen (OpenAI, Azure OpenAI, HuggingFace) en aan memorybackends zoals bv vector stores (Pinecone, Qdrant, Azure AI Search, PostgreSQL, SQLite)

Ze standardiseren interfaces tussen Semantic Kernel agents en de externe services. Hierdoor ontstaat er een soort plug-and-play tussen de verschillende services.

Naast ingebouwde connectors kunnen ook volledig custom connectors worden ontwikkeld.

Memory

Memory in Semantic Kernel is het mechanisme waarmee agents, plugins of applicaties context kunnen opslaan en opvragen. Memory maakt het mogelijk om documenten, feiten, eerdere interacties of andere kennis te hergebruiken tijdens reasoning of planning.

De oorspronkelijke MemoryStore-API bevatte drie varianten:

- Key-value memory: opslag per sleutel met exacte matches, bruikbaar voor voorkeuren of feiten.
- Local file memory: hetzelfde principe als key-value, maar wordt opgeslaan in bestanden en daardoor schaalbaarder.
- Semantic vector memory: tekst wordt omgezet naar embeddings, zodat je kan zoeken op gelijkaardige informatie. Dit was de eerste vorm van “semantic memory” binnen SK.

Deze MemoryStore-architectuur bestaat nog maar is ondertussen legacy en niet meer aanbevolen.

Kernel Memory is de nieuwe architectuur, het is een afzonderlijk project dat Semantic Kernel gebruikt via een connector. Het is een RAG-gebaseerd geheugensysteem: de AI vertrouwt niet enkel op zijn eigen model, maar eerst de informatie gaat opzoeken in externe bronnen voordat het een antwoord genereert. Het gemaakt is voor productieomgevingen en ondersteunt multi-tenant gebruik.

Het ondersteunt functies zoals het importeren van verschillende soorten documenten (webpagina’s, audio, beelden), het opsplitsen in chunks, het genereren van embeddings en het opslaan in vectorstores. Daarnaast kan je data terugvinden via search of volledig opvragen via ask. Queries kunnen worden beperkt met filters op basis van metadata, en via aparte indexes kan je het geheugen opdelen in gescheiden zones.

Door de filtering/tagging mogelijkheden maakt het gecontroleerde multi-tenant toegang mogelijk en hierdoor biedt Kernel Memory veel betere contole op welke data toegangkelijk is voor welke agent of gebruiker, dit zorgt er ook voor dat er veel nauwkeurigere resultaten verkregen worden uit queries.

Search vs Ask

- Search: retourneert de meest relevante memorystukken op basis van vector similarity.
- Ask: voert een volledige RAG-pipeline uit (retrieve → ground → generate) zodat het antwoord gebaseerd is op de gevonden context.

In Semantic Kernel gebruiken agents memory via deze API’s: ze halen context op via Search/Ask en integreren die in hun reasoning of tools. Memory fungeert dus als externe kennisbron, niet als onderdeel van de state zoals bij LangGraph.

Tekortkomingen

Semantic Kernel heeft ook enkele beperkingen. Een eerste punt is dat de ontwikkelactiviteit sinds 2024 sterk gedaald is. De GitHub-activiteit is duidelijk teruggevallen en Microsoft heeft het project officieel in maintenance mode geplaatst. Bugfixes en security updates blijven nog komen, maar structurele vernieuwingen of grote uitbreidingen worden niet langer nagestreefd.

Daarnaast biedt Semantic Kernel geen expliciete workflow- of controlemechanismen. Het framework vertrouwt bijna volledig op automatic function calling. Dat maakt het dynamisch en eenvoudig, maar het zorgt er tegelijk voor dat de volledige controleflow verschuift naar LLM-beslissingen. Developers hebben weinig zicht op de exacte uitvoeringsstappen, en debugging wordt snel complex omdat er geen deterministische of reproduceerbare flow is.

Ook op het vlak van memory duiken er beperkingen op. Kernel Memory is een extern RAG-systeem, wat sterke voordelen heeft voor querying, filtering en multi-tenant governance. Maar omdat het een aparte laag is, introduceert het extra latency en vereist het een complexere configuratie. Voor scenario's waar agents zeer frequente, snelle statusupdates nodig hebben (zoals tijdens korte reasoning-cycli) is deze architectuur minder ideaal. De nauwkeurige filtering en scoping komt dus met een kost: meer componenten, meer configuratie en soms lagere performantie.

Hoewel Semantic Kernel een aantal orkestratiepatronen ondersteunt (zoals sequential, concurrent, handoff en group chat), blijven deze patronen functioneel vrij licht. Ze bieden geen volledige workflow-controle of hiërarchische orchestratie zoals een state machine, planner-agent of centrale coordinator. Complexere patronen moeten nog altijd met MCP of custom logica gebouwd worden.

Use cases

Semantic Kernel is goed in scenario's waar dynamische tool-aanroep, modulaire integraties en eenvoudige agent-automatisatie belangrijk zijn.

- Automatische taakuitvoering via plugins

SK is geschikt voor agents die API-acties moeten uitvoeren op basis van natural-language-instructies, zoals data ophalen, berekeningen doen of workflow-stappen afhandelen zonder vooraf gedefinieerde workflow-graph.

- RAG-gedreven knowledge agents

In combinatie met Kernel Memory kan SK documenten, webpagina's of interne bedrijfsdata ophalen en verwerken. Hierdoor werkt het goed voor vraag en antwoord systemen, documentanalyse of context-gebaseerde assistenten.

- Integraties met bestaande bedrijfssoftware

Door connectors en plugins te gebruiken, kan SK snel verbonden worden met CRM-systemen, databases, interne APIs of cloudservices. Dit maakt het geschikt voor automatisering binnen bestaande software-ecosystemen.

- Parallelle of sequentiële tool-driven workflows

De ondersteunde patterns (sequential, concurrent, group chat, handoff) zijn bruikbaar voor lichtgewicht multi-agent of multi-tool automatisatie in pipelines, zoals data-verrijking of parallelle analyses.

- MCP-gebaseerde multi-tool agents

Dankzij MCP-integratie kan SK een agent laten samenwerken met externe tools of andere agents zonder zelf een orchestrator-laag te moeten bouwen. Dit maakt het bruikbaar voor developer-tools, IDE-assistenten en systeemautomatisatie.

Vergelijking tussen Semantic Kernel en LangGraph

LangGraph

- Deterministiche, stateful workflow-engine: elke stap, branch en stateflow wordt door de developer vastgelegd.
- Sate en context leven in de graaf en worden tussen de nodes doorgegeven.
- Agents zijn strict gebonden aan de structuur van de graaf, waardoor reasoning en geheugen strikt gebonden zijn aan de workflow.
- Geen echte autonomie: een agent kan geen nieuwe plannen, subtaken of tools kiezen buiten de vooraf gedefineerde graaf.
- Geschikt voor controle en herhaalbare bedrijfsprocessen, maar beperkt voor complexe multi-agent dynamiek

Semantic Kernel

- Autonomie via iteratieve function calling: het model beslist zelf welke tools/plugins nodig zijn om een doel te bereiken
- De state/memory zit extern in de Kernel Memory, een schaalbaar RAG-systeem.
- Multi-agent ondersteuning met MCP. Een kernel kan meerdere externe agents, tools of modellen coördineren.
- Modulaire structuur: connectors en plugins die losgekoppeld zijn, developers kunnen verschillende plugins gebruiken of aanmaken en deze uitwisselen zonder veel agentlogica te moeten wijzigen.
- Geschikt voor open taken: planning onstaat tijdens runtime.

Mcp server/client architectuur

Uitbreidingen:
Session lifecycle
Concurrency/parallelisme en error handling
Negotiation op dieper niveau
Vergelijken met andere systemen

MCP introduceert een gestandaardiseerd, bidirectioneel protocol waarin modellen en MCP-servers kunnen communiceren en waarbij een model tools kan aanroepen zonder afhankelijk te zijn van een specifiek AI-framework of vendor. Hierdoor ontstaat een uniforme laag tussen taalmodellen en externe systemen, vergelijkbaar met USB-C in hardware-ecosystemen: één universele aansluiting die het mogelijk maakt uiteenlopende componenten te verbinden.

Dankzij deze standaard wordt het MxN-integratieprobleem opgelost. In plaats van voor elke combinatie van model, framework en databron een aparte integratie te bouwen, fungeert MCP als gemeenschappelijke interface waarop zowel AI-applicaties als externe tools kunnen aansluiten. Dit maakt het mogelijk om tools, databronnen en bedrijfsapplicaties herbruikbaar, uitwisselbaar en schaalbaar te integreren binnen een multi-agentsysteem.

Voor organisaties betekent dit dat generatieve AI op een consistente manier toegang kan krijgen tot interne systemen zoals databases, SharePoint, CRM-platformen of legacy-applicaties zonder nieuwe interfaces te introduceren. Door die standaardisatie ontstaat een toekomstbestendige architectuur waarin agents, LLM's en bedrijfsdata veilig kunnen samenwerken, ongeacht in welke omgeving het wordt gebruikt.

Componenten

De host is de AI-applicatie die de volledige interactie beheert: gebruikersinput, LLM-aanroepen, contextbeheer en de selectie van geschikte tools. De host bevat één of meerdere MCP-clients en bepaalt welke server wanneer wordt aangesproken.

De client is een component binnen de host die een 1-op-1 verbinding onderhoudt met één MCP-server. Het LLM-model communiceert altijd via de client. De client zorgt voor transport, validatie van schema's, en het doorgeven van tool-calls.

Een MCP-server is een extern programma of service die tools, resources en capabilities exposeert volgens het MCP-protocol. De server publiceert zijn mogelijkheden via schema's, ontvangt tool-calls van de client en stuurt gestructureerde resultaten terug

Communicatie flow

1. User input
   De gebruiker geeft een query of intentie aan de host-applicatie.

2. LLM reasoning (in de host)
   De host stuurt de input naar het LLM. Het model beslist welke tool nodig is.

3. Client connectie
   De host stuurt de client aan om verbinding te maken met de juiste MCP-server, als er eerder nog geen verbinding gemaakt werd.

4. Capability Discovery
   De client vraagt de server naar zijn beschikbare tools, parameters, schema's en metadata.

5. Tool Invocation (call_tool)
   De host stuurt een instructie van het model door naar de client, die met de server communiceert volgens het declaratieve schema.

6. Server Execution
   De server voert de gevraagde functionaliteit uit (database query, API-call, berekening, enz.) en stuurt een gestandaardiseerde response terug.

7. Result Delivery
   De client stuurt het resultaat terug naar de host.

8. De host geeft dit:
   - ofwel terug aan het LLM voor verdere reasoning
   - ofwel rechtstreeks aan de gebruiker

9. Iteratie / multi-step workflows
   Het LLM kan meerdere opeenvolgende tool-calls uitvoeren binnen dezelfde context of sessie

Tools

Tijdens de initialisatie van een MCP-verbinding vindt een gestructureerde uitwisseling plaats tussen client en server. De client stuurt eerst een initialize-bericht met daarin de protocolversie, de eigen capabilities en metadata over de implementatie. De server antwoordt daarop met een initialize_result dat zijn beschikbare capabilities bevat, waaronder alle tools, resources en schema's die tijdens de sessie gebruikt kunnen worden. Zodra deze onderhandeling voltooid is, stuurt de client een initialized-notificatie om te bevestigen dat de sessie actief is en dat normale operaties kunnen starten.

Deze fase vormt de basis voor capability negotiation: client en server bepalen welke protocolfuncties tijdens de sessie beschikbaar zijn. Hoewel de meeste informatie tijdens initialisatie wordt uitgewisseld, kan de client op elk moment opnieuw list_capabilities uitvoeren om wijzigingen op te halen, aangezien MCP dynamische updates toelaat.

Tools worden gedeclareerd als onderdeel van de servercapabilities en bevatten altijd een vaste set eigenschappen: name, description, input_schema, output_schema, metadata en eventuele permissions. Deze schema's zijn in JSON Schema-formaat gedefinieerd, waardoor de client parameters strikt kan valideren vóór een tool-call wordt doorgestuurd. Hierdoor ontstaat een veilige en uniforme manier waarop elk model met tools kan werken, ongeacht het onderliggende framework.

MCP: Resources, Tools en Prompts

Binnen het Model Context Protocol (MCP) worden drie soorten server-capabilities onderscheiden.

Resources

Resources zijn data-objecten die een MCP-server aanbiedt aan clients. Dit kunnen bestanden, cloud-objecten (zoals Azure Blob Storage of Amazon S3), databaseitems, web-API-outputs of andere interne bedrijfsgegevens zijn. De client kan resources oplijsten via resources/list, de inhoud ophalen met resources/read en indien ondersteund updates volgen via resources/subscribe. Resources zijn declaratief: het zijn bestaande gegevens die door de client worden beheerd. Ze zijn read-only en voeren geen acties uit.

Tools

Tools vertegenwoordigen uitvoerbare acties op de server. Ze worden door de LLM zelf aangeroepen wanneer het beslist dat een bepaalde actie nodig is. Voorbeelden zijn het versturen van e-mails, browserautomatisatie, bewerkingen in een databank of het uitvoeren van API-calls. Tools zijn imperatief: ze voeren een taak uit op basis van input van het model en geven een strikt gestructureerde output terug volgens vooraf gedefinieerde schema's.

Het belangrijkste onderscheid met resources is dat tools acties uitvoeren en de toestand van externe systemen kunnen wijzigen, terwijl resources enkel bestaande data aanbieden die gelezen kan worden.

Prompts

Prompts zijn vooraf gedefinieerde templates of workflow-instructies die een server kan uitvoeren. Ze maken herbruikbare promptblokken en agentflows beschikbaar aan de host, om complexe interacties te vereenvoudigen. Deze staan los van resources of tools.

Onderliggende communicatie protocollen.

Binnen het Model Context Protocol (MCP) bepaalt de transportlaag hoe JSON-RPC-berichten tussen client en server worden uitgewisseld. MCP ondersteunt daarbij twee ingebouwde transportmechanismen: STDIO en HTTP-gebaseerde streaming.

STDIO gebruikt de standaard input- en outputstreams van een proces. Dit mechanisme is geschikt voor lokale integraties zoals commandline-tools, omdat er geen netwerkverbinding nodig is en de overhead minimaal blijft. De client en server wisselen JSON-RPC-berichten rechtstreeks uit via stdin en stdout.

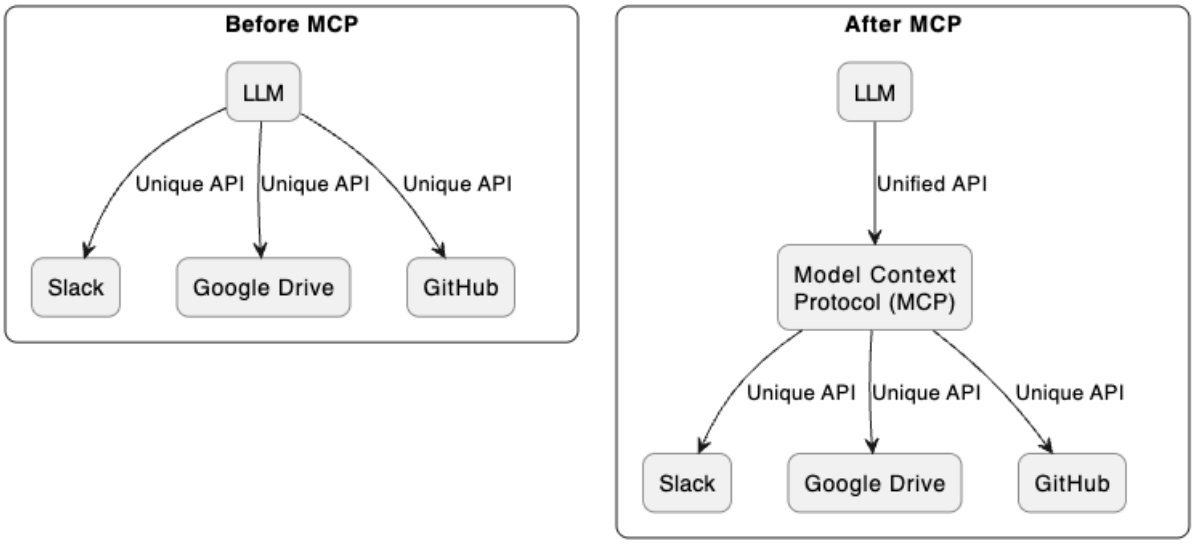
Daarnaast voorziet MCP ook een HTTP-gebaseerd transport dat streaming ondersteunt. In deze configuratie draait de server als een onafhankelijk proces dat meerdere gelijktijdige clients kan bedienen via een HTTP-endpoint. JSON-RPC-berichten van client naar server worden verzonden via HTTP-requests. Voor communicatie van server naar client kan een client bovendien een SSE-stream (Server-Sent Events) openen. Via deze SSE-verbinding kan de server berichten pushen zonder dat de client eerst een verzoek moet sturen, wat streaming en asynchrone notificaties mogelijk maakt.

Deze HTTP-gebaseerde aanpak maakt zowel traditionele request-response interacties als bidirectionele, event-gedreven communicatie mogelijk. MCP specificeert dat servers SSE-events kunnen voorzien van identifiers, zodat door de client gemiste events opnieuw kunnen verstuurd worden of replay-mechanismen kunnen toepassen. Implementaties kunnen daarbij één of meerdere actieve SSE-streams ondersteunen, afhankelijk van serverdesign.

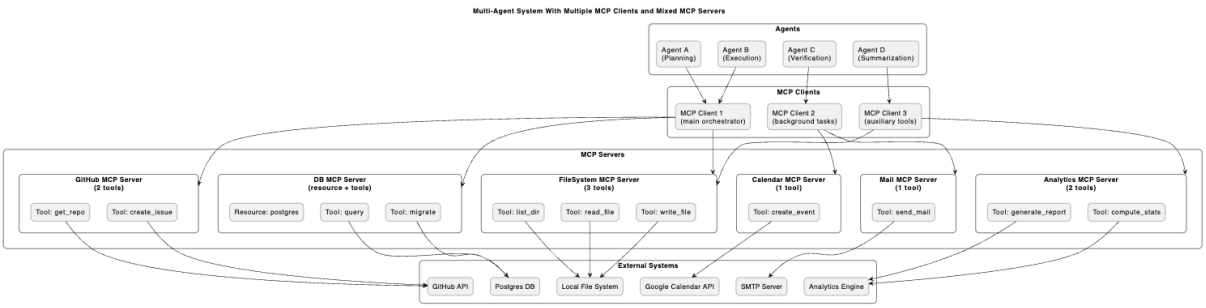
Gevaren

- Verkeerde permissies: een LLM beslist zelf welke tool uitgevoerd wordt. Als de MCP-server niet strict afbakend wie en met welke rechten iets gebeurt, kan een gebruiker onbedoeld acties uitvoeren die niet mogen.
- Lokale MCP-servers voeren OS-commando’s uit. Hierdoor kan er snel commando injectie plaats vinden.
- Een MCP-server kan legitiem lijken, maar later een update pushen die extra data steelt of tools hernoemen zodat een LLM ze misbruikt.
- Sampling laat een MCP-server indirect met een LLM praten. Zonder beperkingen dan een server dit misbruiken om teveel info op te vragen, extra kosten veroorzaken of acties starten, via de client die de LLM-aanroep uitvoert.

Hoe het MCP protocol in multiagentsystemen werkt



Kort fungeert MCP dus als de gestandaardiseerde brugg tussen LLM-agents en de resources / tools die ze aanspreken. Elke agent kan via een MCP-client bepalen welke MCP-servers en bijhoorende tools beschikbaar zijn, en kan zo de bijhorende taken delegeren zonder dat elke agent eigen integraties moet onderhouden.



In dit systeem werken meerdere LLM-agents samen, maar ze gebruiken niet allemaal dezelfde MCP-client. Elke agent voert een specifieke taak uit (planning, executie, verificatie, samenvatten) en stuurt zijn verzoeken naar de MCP-client waaraan hij gekoppeld is. Die clients vormen de toegangspoort tot verschillende MCP-server, waarbij elke client een andere subset van tools en rechten heeft. Het zijn de MCP-servers zelf die de effectieve functionaliteit bevatten (zoals github acties, database operaties, bestandsysteem toegang …) en verbinden met de externe systemen waarin de agents werken.

Bronnen:

Multi agent

- [https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf op 23-10-2025]
- [https://cloud.google.com/discover/what-are-ai-agents 23-10]
- [https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/ 23-10]
- [https://www.youtube.com/watch?v=Xi9Ui-9qcPw&t=381s 23-10]

samenwerking patronen

- [https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns 23-10]
- [Https://strandsagents.com/latest/documentation/docs/user-guide/concepts/multi-agent/multi-agent-patterns/ 23-10]
- [https://www.confluent.io/blog/event-driven-multi-agent-systems/ 23-10]
- [https://medium.com/@princekrampah/multi-agent-architecture-in-multi-agent-systems-multi-agent-system-design-patterns-langgraph-b92e934bf843 19-11]

Hoe context / state gedeeld wordt tussen agents

- [https://medium.com/%40nivalabs.ai/prompt-chaining-for-the-ai-agents-modular-reliable-and-scalable-workflows-a22d15fd5d33 7-11]
- [https://google.github.io/adk-docs/agents/multi-agents/#a-shared-session-state-sessionstate 7-11]
- [https://google.github.io/adk-docs/agents/multi-agents/ #a-shared-session-state-sessionstate 7-11]
- [https://strandsagents.com/latest/documentation/docs/user-guide/concepts/multi-agent/agent-transfer/ 7-11]
- [https://strandsagents.com/latest/documentation/docs/user-guide/concepts/multi-agent/agents-as-tools/7-11]
- [https://microsoft.github.io/autogen/stable/ - agentchat – 7/11]
- [https://strandsagents.com/latest/documentation/docs/user-guide/concepts/multi-agent/blackboard/ 7-11]

Conflict- en consensus afhandeling

- [https://arxiv.org/abs/2407.12532 7-11]
- [https://docs.langchain.com/oss/python/langchain/multi-agent 7-11]
- [https://blog.langchain.com/langgraph-multi-agent-workflows/ 7-11]

Multi-agent frameworks

LangGraph

- [https://www.datacamp.com/blog/best-ai-agents 9-11]
- [https://docs.langchain.com/oss/python/langchain/overview 9-11]
- [https://www.youtube.com/watch?v=1Q_MDOWaljk 9-11]
- [https://medium.com/%40sushmita2310/building-multi-agent-systems-with-langgraph-a-step-by-step-guide-d14088e90f72 14-11]
- [https://blog.langchain.com/how-and-when-to-build-multi-agent-systems 14-11]

- [https://medium.com/%40saeedhajebi/langgraph-is-not-a-true-agentic-framework-3f010c780857 14-11]

Semantic Kernel

- [https://learn.microsoft.com/en-us/semantic-kernel/overview/ 14-11]
- [https://www.youtube.com/watch?v=5JYW_uAxwYM&list=PLPaEUlR-zyKKzhxxTTDATkwiZmbtgt5XY 14-11]
- [19-11 https://medium.com/@sainitesh/exploring-semantic-kernels-multi-agent-orchestration-bfe501ea62d4 19-11]
- [https://learn.microsoft.com/en-us/semantic-kernel/concepts/planning?pivots=programming-language-python 14-11]
- [https://learn.microsoft.com/en-us/semantic-kernel/concepts/plugins/?source=recommendations&pivots=programming-language-python 14-11]
- [https://devblogs.microsoft.com/semantic-kernel/understanding-semantic-kernel-ai-connectors 14-11]

- [https://learn.microsoft.com/en-us/semantic-kernel/concepts/semantic-kernel-components?pivots=programming-language-python 14-11]
- [(https://microsoft.github.io/kernel-memory 18-11]
- [https://venturebeat.com/ai/microsoft-retires-autogen-and-debuts-agent-framework-to-unify-and-govern 19-11]
- [https://thecodestreet.dev/post/what-is-semantic-kernel-best-use-cases-for-building-ai-first-apps-with-microsofts-open-source-sdk/ 19-11]

Mcp Server/client architectuur

- [https://learn.microsoft.com/en-us/azure/developer/ai/intro-agents-mcp 18-11]
- [https://platform.openai.com/docs 18-11]
- [https://github.com/modelcontextprotocol/spec 18-11]
- [https://www.redhat.com/en/blog/model-context-protocol-mcp-understanding-security-risks-and-controls 18-11]

Componenten

- [https://learn.microsoft.com/en-us/azure/developer/ai/intro-agents-mcp 18-11]
- [https://modelcontextprotocol.io/ 18-11]
- Nog dieper gaan into hoe dat werkt + protocollen: https://huggingface.co/learn/mcp-course/unit1/communication-protocol
- [https://huggingface.co/learn/mcp-course/unit1/architectural-components 18-11]
- [https://www.youtube.com/watch?v=TttQxUprbDY 18-11] goated vid
- [https://mcp-framework.com/docs/Transports/http-stream-transport/ 18-11]